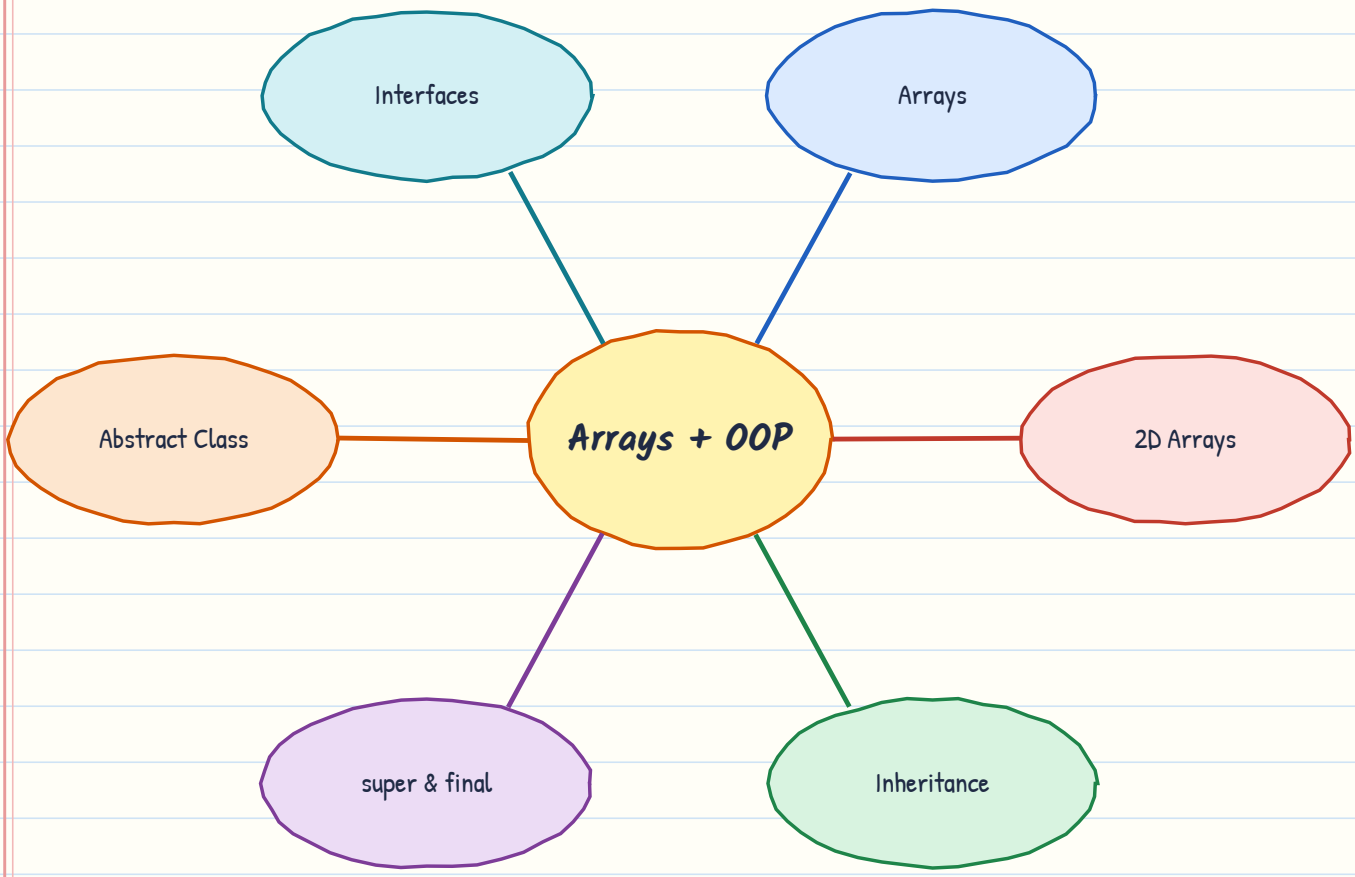
UNIT III

Arrays, Inheritance, Interfaces

1D/2D arrays | Sorting & searching | extends | super | abstract | interface



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

1. Arrays

An ***array*** stores many values of the ***same type*** in one variable, in ***contiguous memory***, accessed by an index that starts at ***0***. Its size is fixed once created.

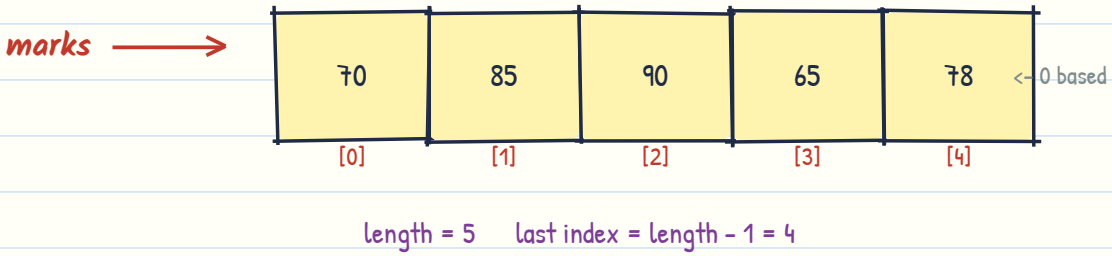


Fig 1: int[] marks = {70, 85, 90, 65, 78}; (stored in heap)

Declaration & initialization

```
int[] a;                        // declare (preferred style)
a = new int[5];                 // create: 5 ints, all 0
int[] b = {10, 20, 30};         // declare + initialize
int c[] = new int[]{1, 2, 3};   // another style
String[] names = new String[3];// default null
```

Array type	Default value
int, long, short, byte	0
double, float	0.0
boolean	false
char	\u0000 (null char)
String / objects	null

Accessing & operations

```java
int[] a = {5, 2, 9, 1};
a[0] = 50;                                  // change element
System.out.println(a.length);               // 4  (length is a field)
int sum = 0, max = a[0];
for (int i = 0; i < a.length; i++) {        // traverse
    sum += a[i];
    if (a[i] > max) max = a[i];
}
for (int x : a) System.out.print(x + " ");  // for-each
```

Careful!

Using index = length (or -1) throws ArrayIndexOutOfBoundsException at run time.

Assigning an array to another

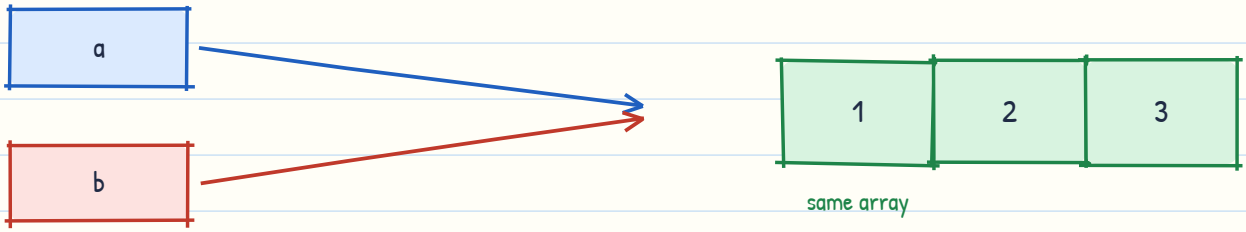


Fig 2: b = a copies the reference; clone() copies the data

```java
int[] a = {1, 2, 3};
int[] b = a;           // alias - change b[0] and a[0] changes
int[] c = a.clone();   // independent copy
int[] d = Arrays.copyOf(a, 5);   // copy + resize -> {1,2,3,0,0}
```

Dynamic change of array size

Arrays cannot grow. To "resize", make a ***bigger new array*** and copy the old values (this is what `ArrayList` does for you).

```java
int[] a = {1, 2, 3};
int[] big = new int[a.length * 2];
System.arraycopy(a, 0, big, 0, a.length);
a = big;               // a now has 6 slots
```

Sorting & searching

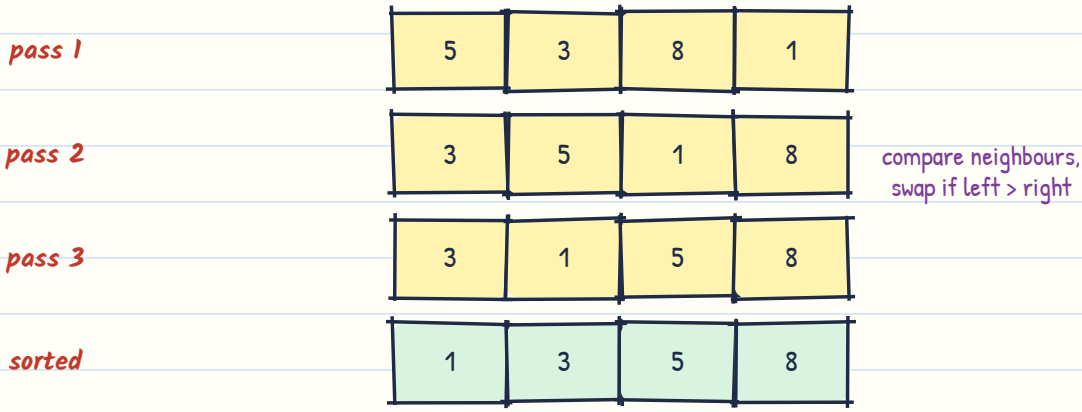


Fig 3: Bubble sort – biggest value "bubbles" to the end

Bubble sort

```
for (int i = 0; i < n - 1; i++)            // passes
    for (int j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1]) {              // swap
            int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
        }
```

Binary search

```
int low = 0, high = a.length - 1;          // Binary search (array must be SORTED)
while (low <= high) {
    int mid = (low + high) / 2;
    if (a[mid] == key)      return mid;
    else if (a[mid] < key)  low = mid + 1;
    else                    high = mid - 1;
}
return -1;                                  // not found
```

Class Arrays (java.util.Arrays)

Method	Purpose
Arrays.sort(a)	sort ascending
Arrays.binarySearch(a,key)	search in sorted array
Arrays.toString(a)	print as [1, 2, 3]
Arrays.fill(a, v)	fill with value
Arrays.equals(a, b)	compare contents
Arrays.copyOf(a, len)	copy / resize

Two-dimensional arrays

An array of arrays - think of a *table* with rows and columns: `a[row][col]`.

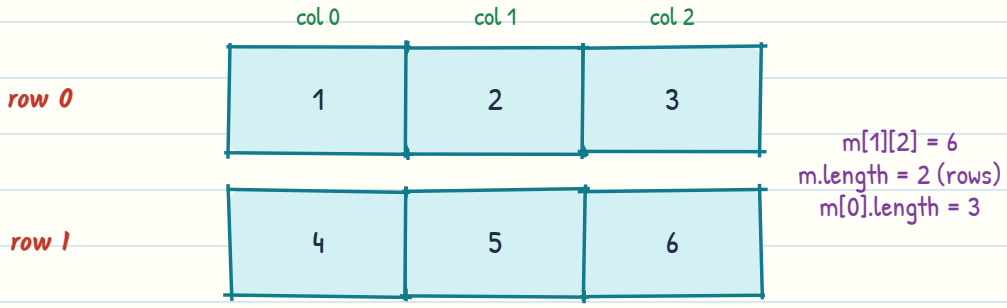


Fig 4: int[][] m = { {1,2,3}, {4,5,6} };

```
int[][] m = new int[2][3];
int[][] t = { {1,2,3}, {4,5,6} };
for (int i = 0; i < t.length; i++) {
    for (int j = 0; j < t[i].length; j++)
        System.out.print(t[i][j] + " ");
    System.out.println();
}
```

```
> OUTPUT
1 2 3
4 5 6
```

Arrays of varying lengths (jagged) & 3D arrays

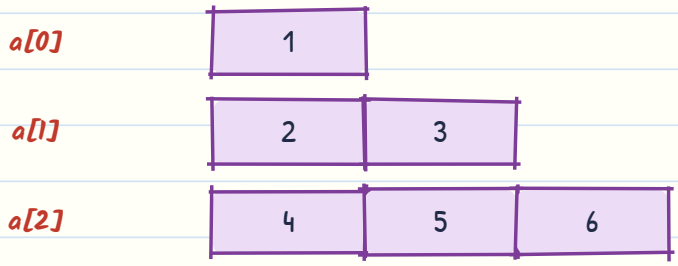


Fig 5: Jagged array - each row has its own length

```
int[][] j = new int[3][];        // only rows created
j[0] = new int[1];  j[1] = new int[2];  j[2] = new int[3];
int[][][] cube = new int[2][3][4];   // 3D: 2 blocks, 3 rows, 4 cols
```

Arrays as vectors: a 1-D array is a vector; for growable vectors use `java.util.Vector` or `ArrayList` (`add()`, `get()`, `remove()`, `size()`).

2. Inheritance

Inheritance lets a class (***subclass / child***) acquire fields and methods of another (***superclass / parent***) using **extends**. It gives ***code reuse*** and an ***IS-A*** relationship.

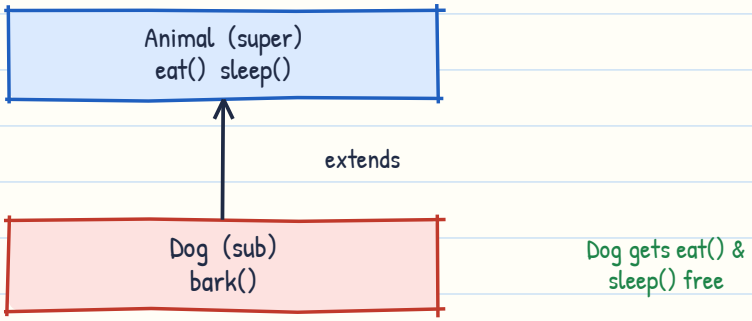


Fig 6: Dog IS-A Animal

```java
class Animal {
    void eat() { System.out.println("eating"); }
}
class Dog extends Animal {
    void bark() { System.out.println("barking"); }
}
public class Test {
    public static void main(String[] a) {
        Dog d = new Dog();
        d.eat();  d.bark();       // eat() inherited
    }
}
```

```
> OUTPUT
eating
barking
```

Types of inheritance

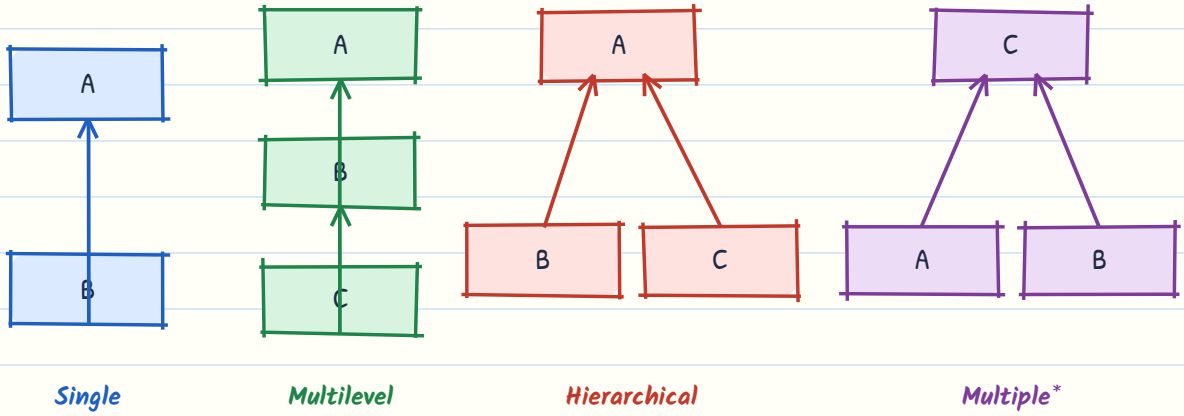


Fig 7: Types of inheritance

> **Important**
>
> *Java does NOT allow multiple inheritance with classes (diamond problem / ambiguity). It is achieved using INTERFACES. Hybrid = combination, also via interfaces.

Universal superclass - Object class

Every Java class ***automatically extends java.lang.Object*** - it is the root of the class tree.

So all objects have these methods:

Method	Purpose
toString()	text form of object
equals(obj)	compare objects
hashCode()	integer hash
getClass()	runtime class info

Multilevel inheritance & constructors

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
class C extends B { C() { System.out.println("C"); } }
new C();     // constructors run TOP -> DOWN
```

```
> OUTPUT
A
B
C
```

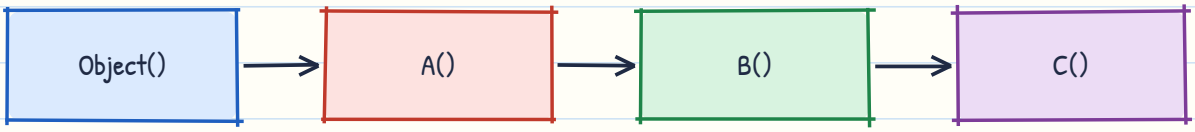


Fig 8: Constructor order parent first, child last

Keyword super

- `super.member` - access parent's field/method hidden by child.
- `super(args)` - call parent constructor; must be the *first* statement.
- Without it Java inserts `super();` automatically.

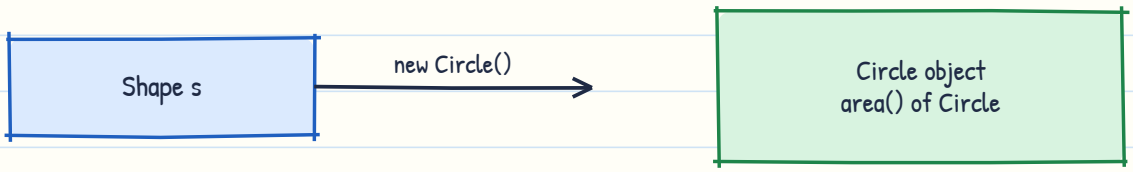
```
class Person { String name; Person(String n) { name = n; }
               void show() { System.out.println("Person " + name); } }
class Student extends Person {
    int roll;
    Student(String n, int r) { super(n); roll = r; }   // parent constructor
    void show() { super.show(); System.out.println("Roll " + roll); }
}
```

Final, access control & inheritance

- `final class` cannot be extended; `final` method cannot be overridden.
- `private` members are NOT inherited (use getters); `protected` & `public` are.
- Subclass can *widen* but not narrow access when overriding.

Dynamic method dispatch (runtime polymorphism)

A *parent reference can hold a child object*; the method that runs is decided at *run time* by the actual object.



s.area() -> JVM looks at the OBJECT, runs Circle.area()

Fig 9: Parent ref -> child object

```
class Shape  { void draw() { System.out.println("shape"); } }
class Circle extends Shape { void draw() { System.out.println("circle"); } }
class Square extends Shape { void draw() { System.out.println("square"); } }
Shape s;
s = new Circle();  s.draw();     // circle
s = new Square();  s.draw();     // square
```

Abstract classes

An ***abstract class*** is incomplete – it may have abstract methods (no body) that children ***must*** implement. You cannot create its object.

```
abstract class Shape {
    abstract double area();                  // no body
    void info() { System.out.println("I am a shape"); }   // normal method OK
}
class Rect extends Shape {
    double w, h;  Rect(double w, double h) { this.w = w; this.h = h; }
    double area() { return w * h; }          // must implement
}
// Shape s = new Shape();  ERROR      Shape s = new Rect(2,3);  OK
```

3. Interfaces

An ***interface*** is a 100% abstract contract: it lists *what* a class must do, not *how*. A class ***implements*** it. This gives ***multiple inheritance*** and loose coupling.

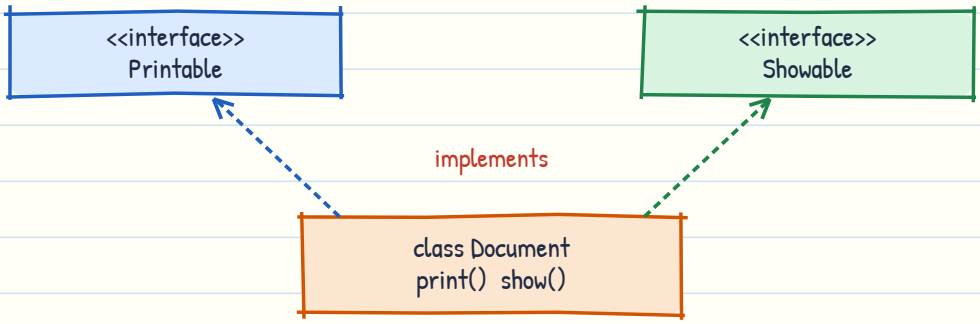


Fig 10: class implements two interfaces (multiple inheritance of type)

```java
interface Printable {                  // declaration
    int MAX = 10;                      // public static final (constant)
    void print();                      // public abstract
}
interface Showable { void show(); }
class Doc implements Printable, Showable {      // multiple interfaces
    public void print() { System.out.println("printing"); }  // must be public
    public void show()  { System.out.println("showing"); }
}
```

	Abstract class	Interface
Keyword	extends (one only)	implements (many)
Methods	abstract + normal	abstract (+default/static)
Variables	any kind	only constants
Constructor	yes	no

Nested interfaces & inheritance of interfaces

```java
interface A { void a(); }
interface B extends A { void b(); }       // interface extends interface
class C implements B { public void a(){}  public void b(){} }  // must do both
class Outer { interface Inner { void f(); } }   // nested interface
```

Default & static methods (Java 8)

```java
interface Vehicle {
    void run();
    default void honk() { System.out.println("beep"); }   // has a body
    static void rules() { System.out.println("drive safe"); }
}
Vehicle.rules();     // call static method with interface name
```

Functional interfaces & annotations

A *functional interface* has exactly *one abstract method*; it can be written as a short *lambda* expression. `@FunctionalInterface` makes the compiler check it.

```java
@FunctionalInterface
interface Calc { int op(int a, int b); }
Calc add = (a, b) -> a + b;          // lambda
Calc mul = (a, b) -> a * b;
System.out.println(add.op(3, 4));    // 7
```

```
> OUTPUT
7
```

Annotation	Meaning
@Override	method overrides parent - compiler checks
@FunctionalInterface	exactly one abstract method
@Deprecated	old, avoid using
@SuppressWarnings	hide compiler warnings

Quick Revision & Exam Questions

Topic	Memory line
Array	same type, fixed size, index from 0, length field
a = b	alias; use clone() / copyOf() to copy
extends	one parent only for classes
super	parent member / super(...) first line
Dispatch	parent ref, child object, run-time choice
abstract	cannot instantiate; abstract methods forced
interface	all abstract, implements many, default/static allowed

Likely exam questions

- Explain declaration, initialization and traversal of arrays with a program.
- Write a program for bubble sort / binary search.
- Explain 2-D and jagged arrays with diagrams.
- Explain types of inheritance with examples; why no multiple inheritance?
- Explain keyword super and constructor order in inheritance.
- What is dynamic method dispatch? Explain with a program.
- Differentiate abstract class and interface.
- Explain default, static and functional interfaces with examples.